

Cloud Infrastructure Automation using Terraform and AWS

Services: EC2, RDS, VPC, Terraform, Application Load Balancer (ALB)

Description: Automate the provisioning of a complete AWS application infrastructure using Terraform. The infrastructure includes EC2 instances inside a custom VPC with public and private subnets, an Application Load Balancer, and an Amazon RDS MySQL database.

Skills: Infrastructure as Code (IaC), Terraform, VPC Setup, AWS Networking, EC2 Deployment, RDS Configuration, Load Balancer Configuration.

Definition: Cloud Infrastructure Automation using Terraform and AWS is designed to automate the deployment and management of AWS cloud resources using Infrastructure as Code (IaC). Terraform is used to provision AWS services such as VPC, subnets, EC2 instances, Application Load Balancer, and RDS database automatically through configuration files. The project reduces manual configuration effort, improves consistency, and enables faster infrastructure deployment in AWS environments.

Scope of the Project: The scope of this project is to design and implement an automated AWS cloud infrastructure deployment using Terraform.

This project covers:

- Automated Infrastructure Provisioning using Terraform.
- Creation of Custom VPC with Public and Private Subnets.
- Deployment of EC2 Web Server.
- Configuration of Application Load Balancer.
- Deployment of Amazon RDS MySQL Database.
- Security Group and Route Table Configuration.
- Infrastructure Automation using Infrastructure as Code (IaC).

Methodology:

1. Terraform + AWS Services – Best Method

- Uses Terraform to automate AWS infrastructure deployment.
- Creates complete infrastructure using code.
- Provides reusable and scalable infrastructure.
- Reduces manual configuration errors.
- Supports Infrastructure as Code (IaC) best practices.
- Production-oriented cloud deployment approach.
- Easy to modify and redeploy infrastructure.
- Suitable for cloud automation projects.

2. Manual AWS Console Deployment – Alternative Method

- Infrastructure resources are created manually using AWS Console.
- Requires manual configuration of networking and security.
- Time-consuming and less scalable.
- Difficult to maintain for large infrastructure deployments.
- Higher chances of configuration mistakes.
- Not ideal for modern DevOps practices.

AWS Services used in the Project:

1. Amazon EC2 (Elastic Compute Cloud)

Definition: Amazon EC2 is a cloud computing service that provides scalable virtual servers in AWS.

Purpose: Hosts the web application server.

Role:

- Runs Apache2 web server.
- Hosts application webpage.
- Receives traffic from Load Balancer.

2. Amazon RDS (Relational Database Service)

Definition: Amazon RDS is a managed relational database service provided by AWS.

Purpose: Provides managed MySQL database service.

Role:

- Stores application database.
- Runs inside private subnet for security.
- Provides secure backend database connectivity.

3. Amazon VPC (Virtual Private Cloud)

Definition: Amazon VPC enables users to create isolated virtual networks in AWS.

Purpose: Provides secure network infrastructure.

Role:

- Creates public and private subnets.
- Controls routing and internet access.
- Isolates AWS resources securely.

4. Application Load Balancer (ALB)

Definition: Application Load Balancer distributes incoming application traffic across targets.

Purpose: Balances incoming HTTP traffic.

Role:

- Receives user requests.
- Routes traffic to EC2 web server.
- Improves availability and scalability.

5. Terraform

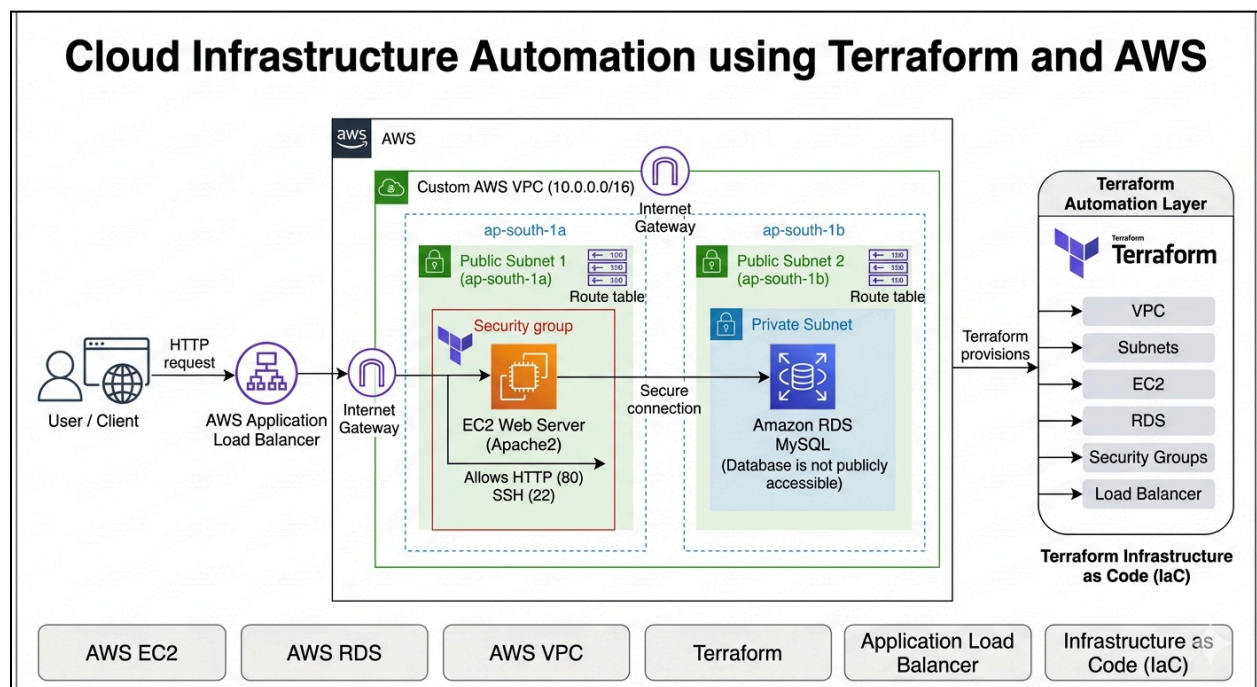
Definition: Terraform is an Infrastructure as Code (IaC) tool used to automate cloud infrastructure deployment.

Purpose: Automates AWS resource provisioning.

Role:

- Creates VPC, EC2, RDS, and ALB automatically.
- Maintains infrastructure using code.
- Simplifies cloud deployment and management.

Architecture Diagram:

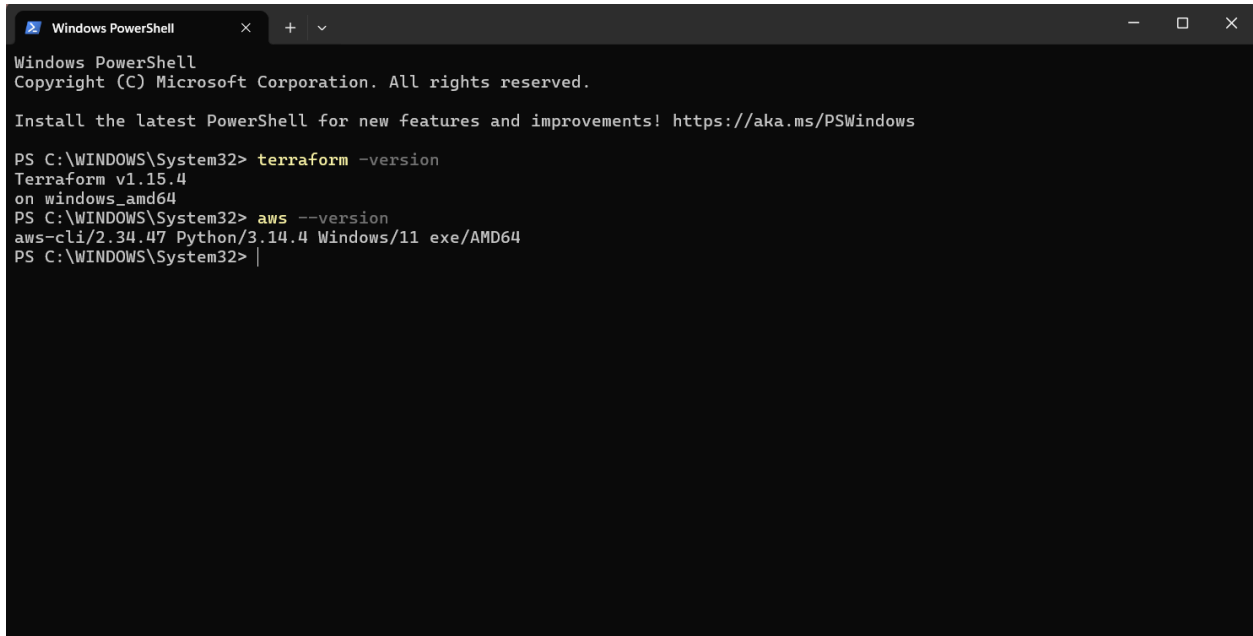


Implementation Steps to Do the Project:

Step 1: Install Terraform

- Download Terraform from official website.
- Install Terraform in local system.
- Verify installation using: `terraform -version`

Cloud Infrastructure Automation using Terraform and AWS



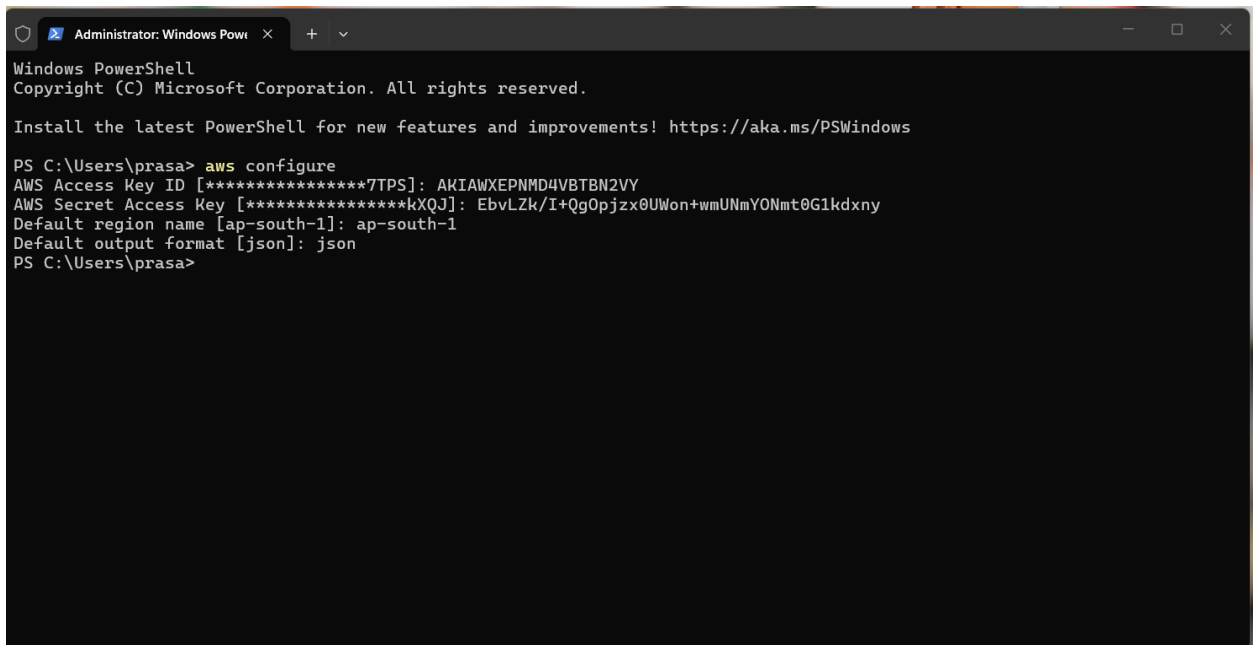
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\System32> terraform -version
Terraform v1.15.4
on windows_amd64
PS C:\WINDOWS\System32> aws --version
aws-cli/2.34.47 Python/3.14.4 Windows/11 exe/AMD64
PS C:\WINDOWS\System32> |
```

Step 2: Configure AWS CLI

- Install AWS CLI.
- Configure AWS credentials using: aws configure
- Enter: Access Key, Secret Key, Region: ap-south-1



```
Administrator: Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

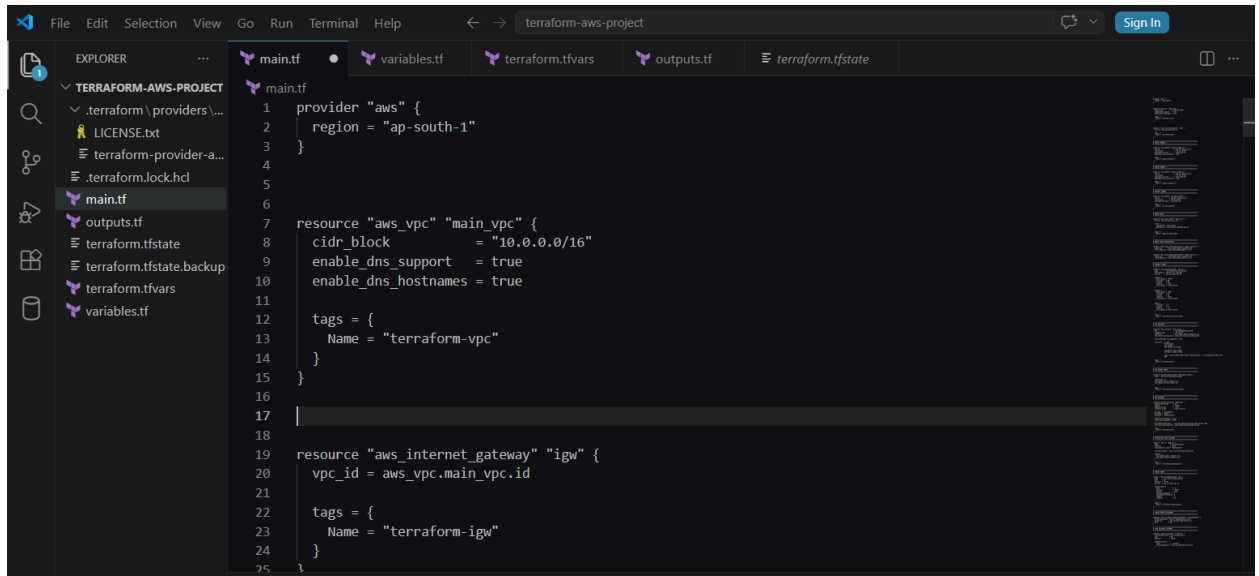
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\prasa> aws configure
AWS Access Key ID [*****7TPS]: AKIAWXPND4VBTBN2VY
AWS Secret Access Key [*****kXQJ]: EbvLZk/I+QgOpjzx0UWon+wmUNmYONmt0G1kdxny
Default region name [ap-south-1]: ap-south-1
Default output format [json]: json
PS C:\Users\prasa>
```

Step 3: Create Terraform Project Files

Create files:

- main.tf
- variables.tf
- terraform.tfvars
- outputs.tf

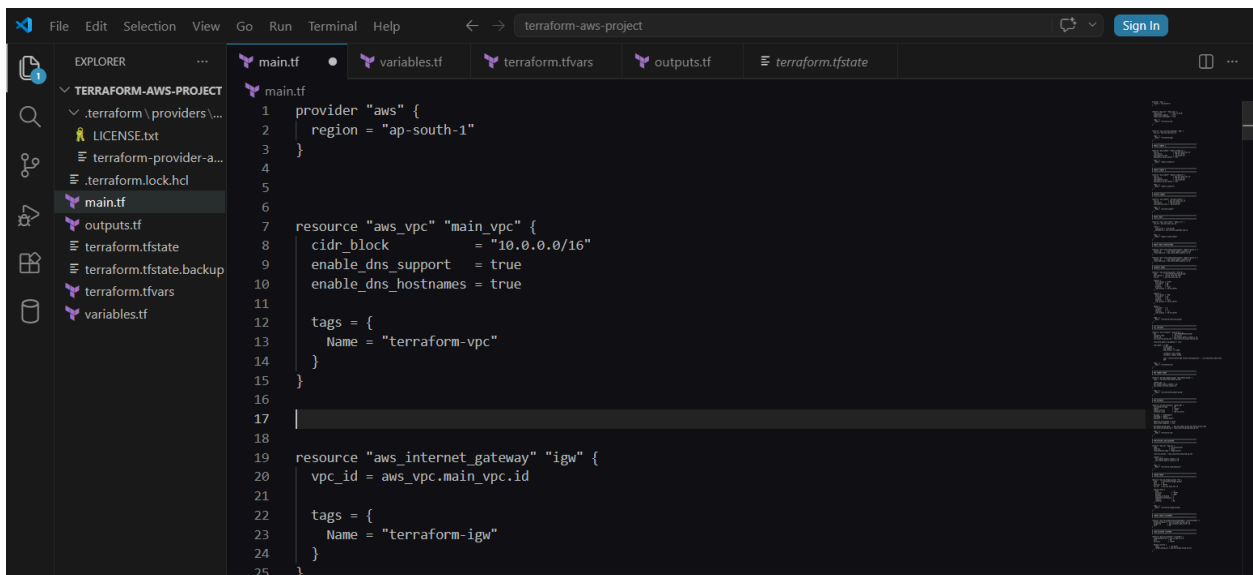


The screenshot shows the Visual Studio Code editor with a project named 'terraform-aws-project'. The Explorer sidebar on the left shows the file structure: 'TERRAFORM-AWS-PROJECT' containing '.terraform/providers/...', 'LICENSE.txt', 'terraform-provider-a...', 'terraform.lock.hcl', 'main.tf', 'outputs.tf', 'terraform.tfstate', 'terraform.tfstate.backup', 'terraform.tfvars', and 'variables.tf'. The main editor displays the 'main.tf' file with the following Terraform code:

```
1 provider "aws" {
2   region = "ap-south-1"
3 }
4
5
6
7 resource "aws_vpc" "main_vpc" {
8   cidr_block      = "10.0.0.0/16"
9   enable_dns_support = true
10  enable_dns_hostnames = true
11
12  tags = {
13    Name = "terraform-vpc"
14  }
15 }
16
17
18
19 resource "aws_internet_gateway" "igw" {
20   vpc_id = aws_vpc.main_vpc.id
21
22   tags = {
23     Name = "terraform-igw"
24   }
25 }
```

Step 4: Create VPC and Subnets using Terraform

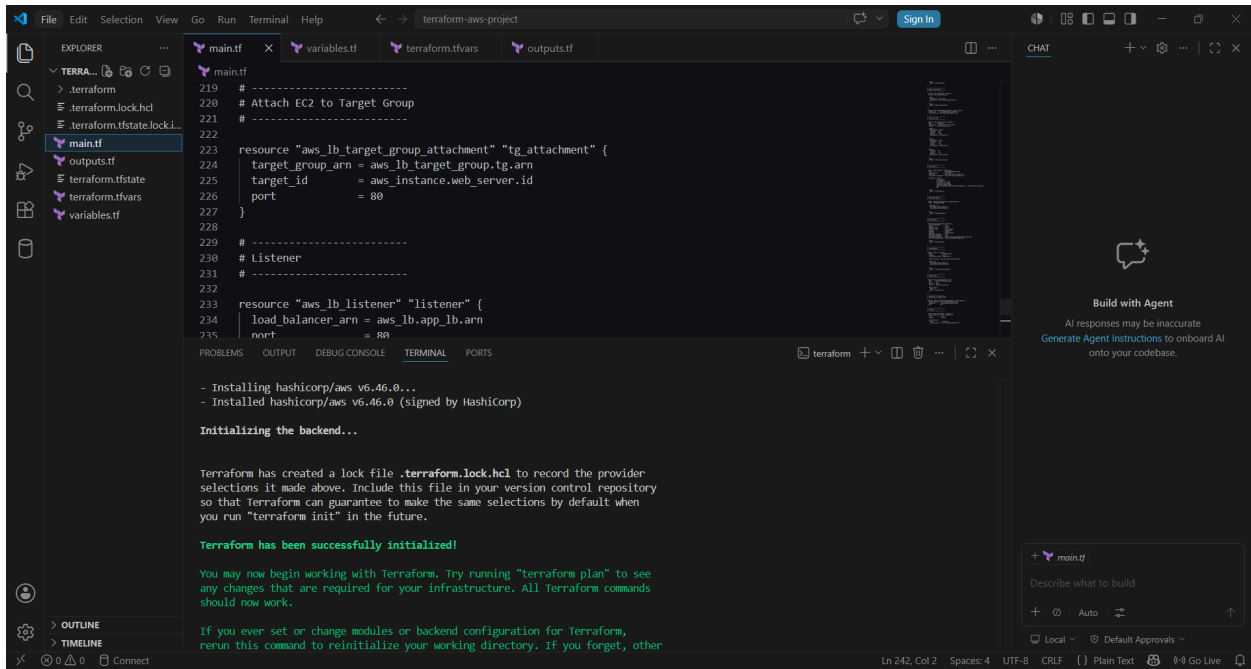
- Define custom VPC.
- Create public subnet for EC2.
- Create private subnet for RDS.
- Configure Internet Gateway and Route Tables.



This screenshot is identical to the one above, showing the Visual Studio Code editor with the 'main.tf' file. The file structure in the Explorer sidebar and the Terraform code in the main editor are the same as in the previous image.

Run Terraform commands:

- terraform init
- terraform plan
- terraform apply



The screenshot shows the Visual Studio Code interface with a project named 'terraform-aws-project'. The Explorer pane on the left shows the file structure: .terraform, .terraform.lock.hcl, main.tf, outputs.tf, terraform.tfstate, terraform.tfvars, and variables.tf. The main editor displays the content of 'main.tf', which defines an AWS Lambda target group attachment and a listener. The terminal pane at the bottom shows the output of the 'terraform init' command. It indicates that the HashiCorp Terraform CLI v6.46.0 is being installed and that the backend has been successfully initialized. A message states: 'Terraform has been successfully initialized! You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.' A sidebar on the right shows a 'Build with Agent' section with a chat interface.

```
219 # -----
220 # Attach EC2 to Target Group
221 # -----
222
223 resource "aws_lb_target_group_attachment" "tg_attachment" {
224   target_group_arn = aws_lb_target_group.tg.arn
225   target_id        = aws_instance.web_server.id
226   port             = 80
227 }
228
229 # -----
230 # Listener
231 # -----
232
233 resource "aws_lb_listener" "listener" {
234   load_balancer_arn = aws_lb_app_lb.arn
235   port              = 80
236 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
- Installing hashicorp/aws v6.46.0...
- Installed hashicorp/aws v6.46.0 (signed by HashiCorp)

Initializing the backend...

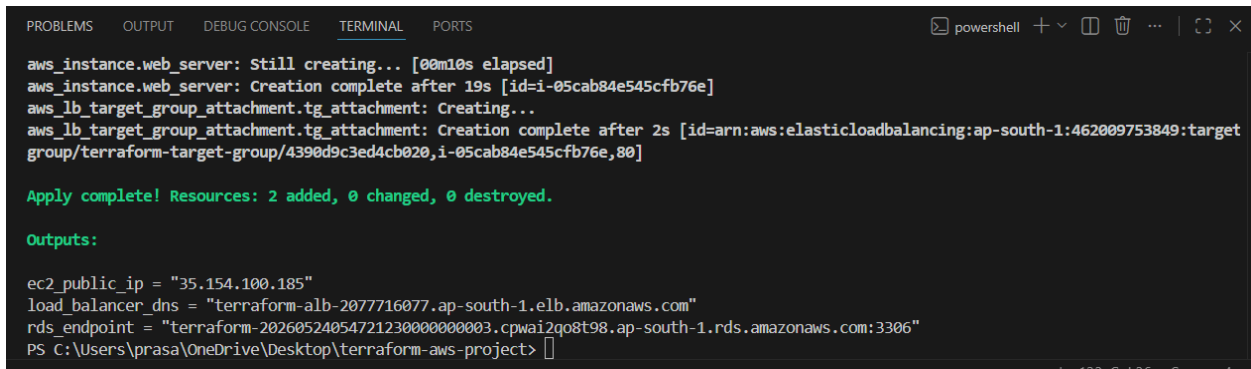
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
```

Ln 242, Col 2, Spaces: 4 UTF-8 CRLF Plain Text Go Live



The screenshot shows the terminal output of the 'terraform apply' command. It displays the progress of creating and attaching resources. The output shows that the EC2 instance is still creating, the target group attachment is creating, and the listener is creating. The final output shows that the apply was successful, with 2 resources added and 0 changed or destroyed. The outputs section shows the public IP of the EC2 instance, the DNS name of the load balancer, and the endpoint of the RDS instance.

```
aws_instance.web_server: Still creating... [00m10s elapsed]
aws_instance.web_server: Creation complete after 19s [id=i-05cab84e545cfb76e]
aws_lb_target_group_attachment.tg_attachment: Creating...
aws_lb_target_group_attachment.tg_attachment: Creation complete after 2s [id=arn:aws:elasticloadbalancing:ap-south-1:462009753849:target
group/terraform-target-group/4390d9c3ed4cb020,i-05cab84e545cfb76e,80]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

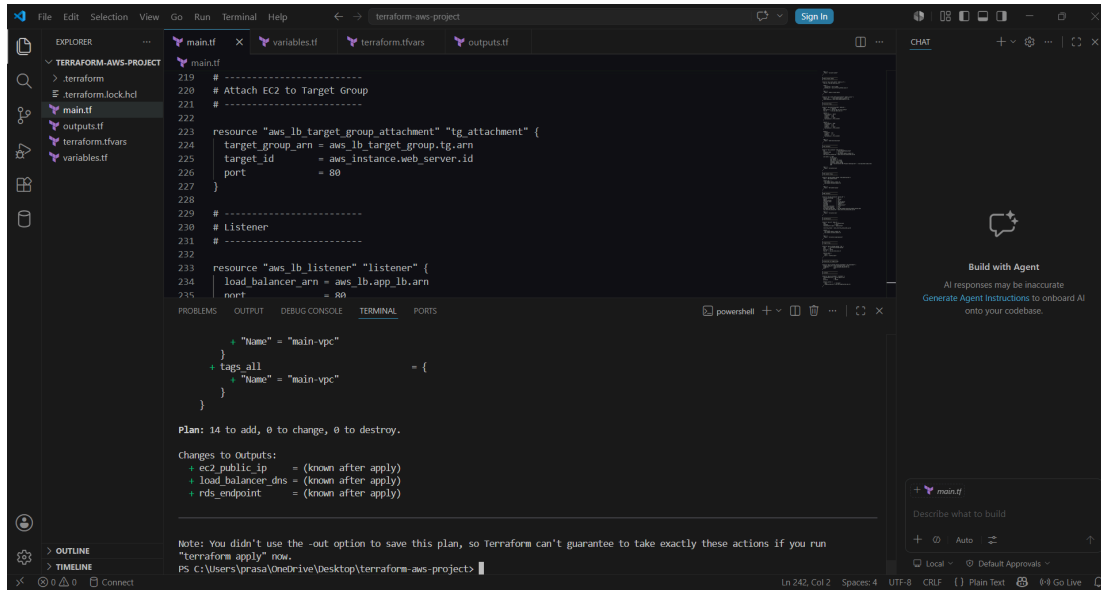
ec2_public_ip = "35.154.100.185"
load_balancer_dns = "terraform-alb-2077716077.ap-south-1.elb.amazonaws.com"
rds_endpoint = "terraform-20260524054721230000000003.cpwai2qo8t98.ap-south-1.rds.amazonaws.com:3306"
PS C:\Users\prasa\OneDrive\Desktop\terraform-aws-project>
```

Ln 122, Col 26, Spaces: 4

Step 5: Create Security Groups

- Create security group for EC2 instance.
- Allow HTTP (80) and SSH (22).
- Create security group for RDS.
- Allow MySQL port 3306 only from EC2 security group.

Cloud Infrastructure Automation using Terraform and AWS

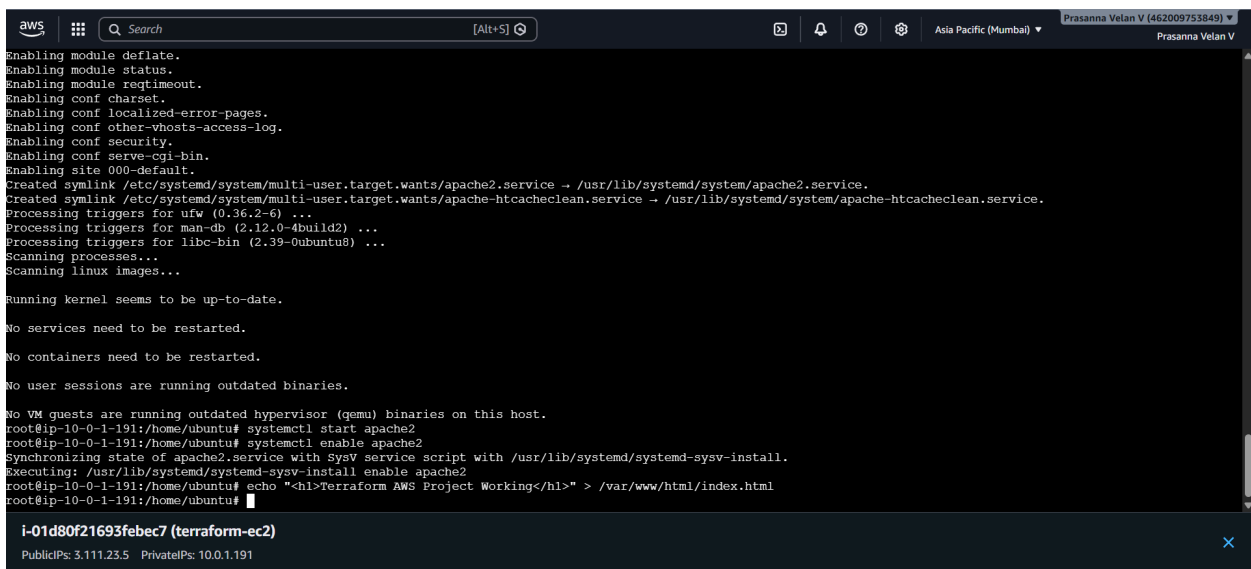


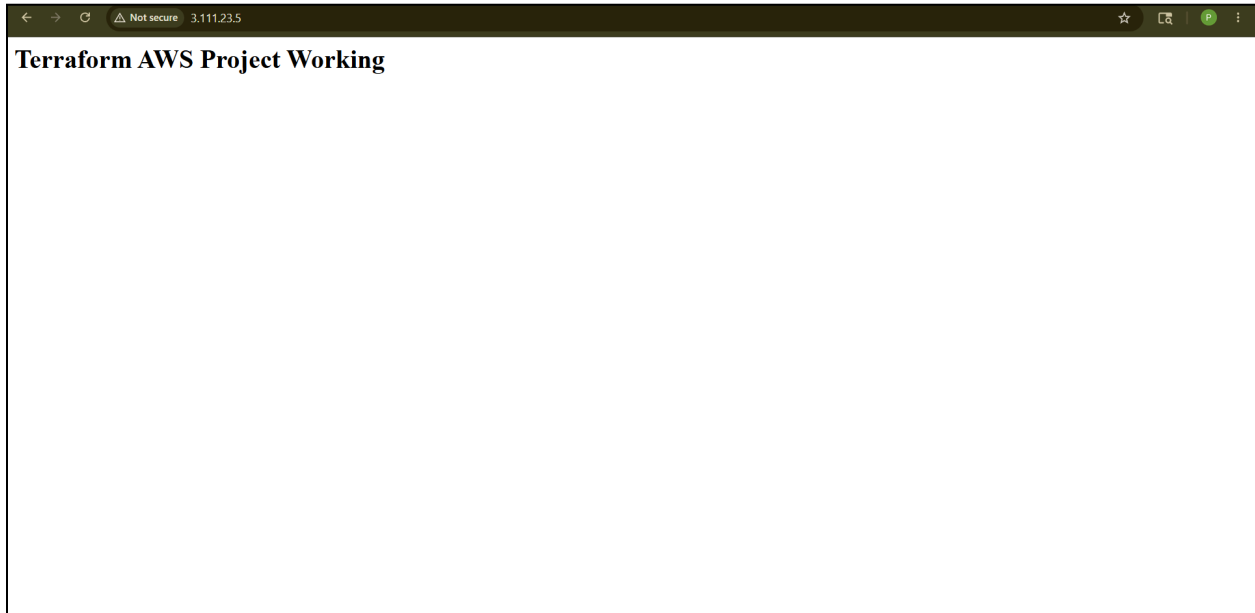
Step 6: Launch EC2 Instance

- Use Terraform to create EC2 web server.
- Install Apache web server.
- Deploy sample web application.

Verify:

- Copy EC2 public IP.
- Open browser and check web server response.





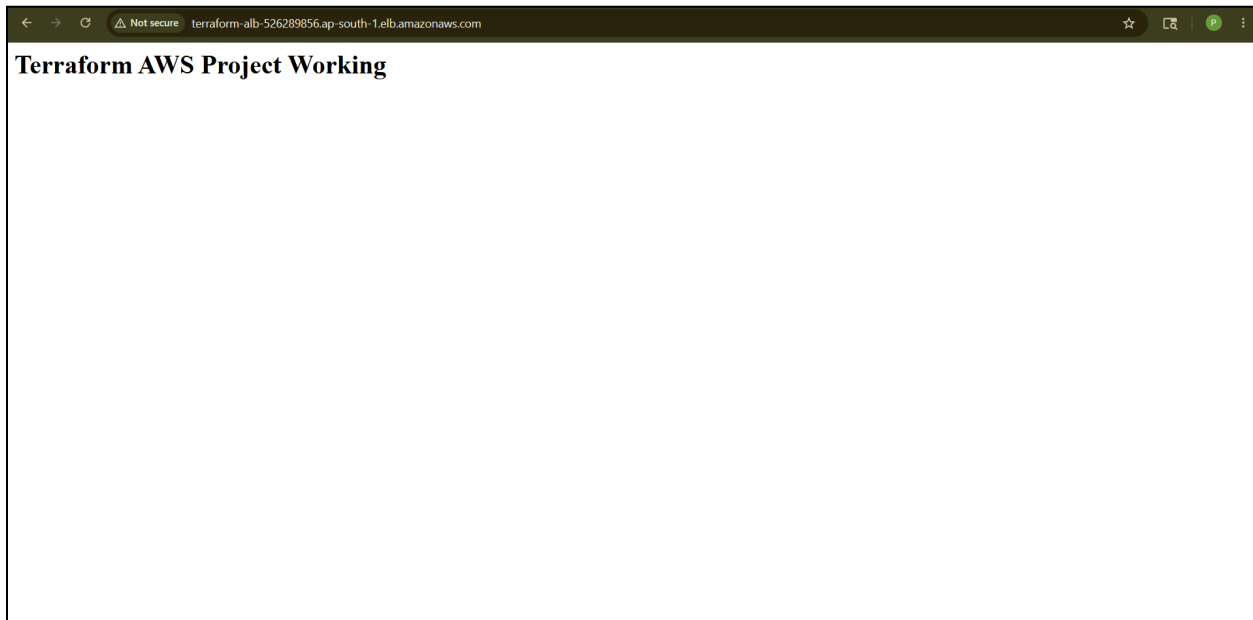
Step 7: Create Application Load Balancer

- Create Target Group.
- Attach EC2 instance to Target Group.
- Configure Application Load Balancer listener.
- Verify ALB DNS endpoint in browser.

```

aws_instance.web_server: Creating...
aws_route_table_association.public_assoc_2: Creation complete after 0s [id=rtbassoc-08e9e16c37e056a37]
aws_route_table_association.public_assoc_1: Creation complete after 0s [id=rtbassoc-06931592eb2c64789]
aws_lb.app_lb: Still creating... [00m10s elapsed]
aws_instance.web_server: Still creating... [00m10s elapsed]
aws_instance.web_server: Creation complete after 13s [id=i-01d80f21693feb7c7]
aws_lb_target_group_attachment.tg_attachment: Creating...
aws_lb_target_group_attachment.tg_attachment: Creation complete after 0s [id=arn:aws:elasticloadbalancing:ap-south-1:462009753849:target-group/terraform-target-group/84d41a747726514f,i-01d80f21693feb7c7,80]
aws_lb.app_lb: Still creating... [00m20s elapsed]
aws_lb.app_lb: Still creating... [00m30s elapsed]
aws_lb.app_lb: Still creating... [00m40s elapsed]
aws_lb.app_lb: Still creating... [00m50s elapsed]
aws_lb.app_lb: Still creating... [01m00s elapsed]
aws_lb.app_lb: Still creating... [01m10s elapsed]
aws_lb.app_lb: Still creating... [01m20s elapsed]
aws_lb.app_lb: Still creating... [01m30s elapsed]
aws_lb.app_lb: Still creating... [01m40s elapsed]
aws_lb.app_lb: Still creating... [01m50s elapsed]
aws_lb.app_lb: Still creating... [02m00s elapsed]
aws_lb.app_lb: Still creating... [02m10s elapsed]
aws_lb.app_lb: Still creating... [02m20s elapsed]
aws_lb.app_lb: Still creating... [02m30s elapsed]
aws_lb.app_lb: Still creating... [02m40s elapsed]
aws_lb.app_lb: Still creating... [02m50s elapsed]
aws_lb.app_lb: Creation complete after 2m56s [id=arn:aws:elasticloadbalancing:ap-south-1:462009753849:loadbalancer/app/terraform-alb/fa4430fa806ce544]
aws_lb_listener.listener: Creating...
aws_lb_listener.listener: Creation complete after 1s [id=arn:aws:elasticloadbalancing:ap-south-1:462009753849:listener/app/terraform-alb/fa4430fa806ce544/863b189e93ec7d1a1]

```

Step 8: Create Amazon RDS Database

- Create MySQL RDS instance using Terraform.
- Deploy database inside private subnet.
- Configure DB subnet group.
- Verify database connectivity from EC2 instance.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

aws_db_instance.mysql_db: Creating...
aws_db_instance.mysql_db: Still creating... [00m10s elapsed]
aws_db_instance.mysql_db: Still creating... [00m20s elapsed]
aws_db_instance.mysql_db: Still creating... [00m30s elapsed]
aws_db_instance.mysql_db: Still creating... [00m40s elapsed]
aws_db_instance.mysql_db: Still creating... [00m50s elapsed]
aws_db_instance.mysql_db: Still creating... [01m00s elapsed]
aws_db_instance.mysql_db: Still creating... [01m10s elapsed]
aws_db_instance.mysql_db: Still creating... [01m20s elapsed]
aws_db_instance.mysql_db: Still creating... [01m30s elapsed]
aws_db_instance.mysql_db: Still creating... [01m40s elapsed]
aws_db_instance.mysql_db: Still creating... [01m50s elapsed]
aws_db_instance.mysql_db: Still creating... [02m00s elapsed]
aws_db_instance.mysql_db: Still creating... [02m10s elapsed]
aws_db_instance.mysql_db: Still creating... [02m20s elapsed]
aws_db_instance.mysql_db: Still creating... [02m30s elapsed]
aws_db_instance.mysql_db: Still creating... [02m40s elapsed]
aws_db_instance.mysql_db: Still creating... [02m50s elapsed]
aws_db_instance.mysql_db: Still creating... [03m00s elapsed]
aws_db_instance.mysql_db: Still creating... [03m10s elapsed]
aws_db_instance.mysql_db: Still creating... [03m20s elapsed]
aws_db_instance.mysql_db: Still creating... [03m30s elapsed]
aws_db_instance.mysql_db: Still creating... [03m40s elapsed]
aws_db_instance.mysql_db: Still creating... [03m50s elapsed]
aws_db_instance.mysql_db: Still creating... [04m00s elapsed]
aws_db_instance.mysql_db: Still creating... [04m10s elapsed]
aws_db_instance.mysql_db: Still creating... [04m20s elapsed]
aws_db_instance.mysql_db: Still creating... [04m30s elapsed]
aws_db_instance.mysql_db: Creation complete after 4m38s [id=db-GUJOZPU3DFXC65TJ6DZBAHKLMW]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Step 9: Verify Terraform Infrastructure

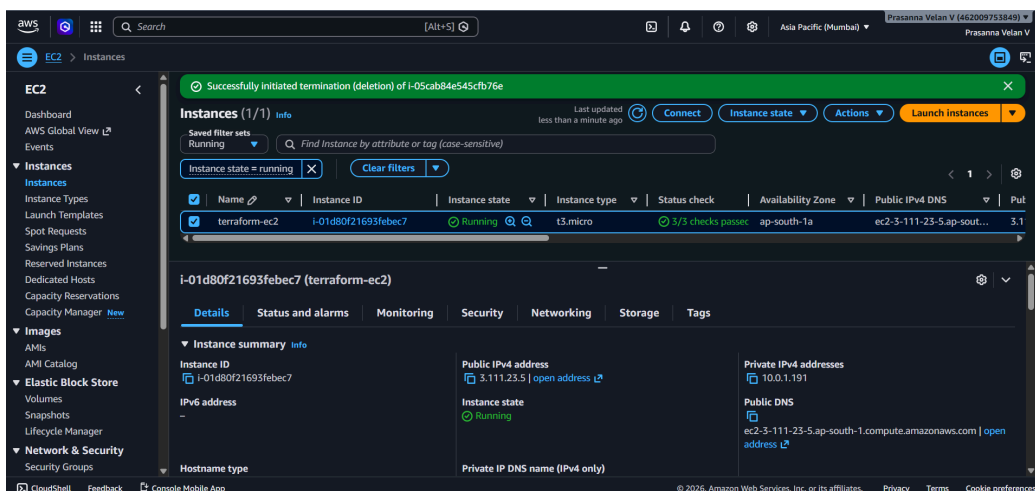
Run: terraform state list

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
aws_db_instance.mysql_db: Still creating... [04m00s elapsed]
aws_db_instance.mysql_db: Still creating... [04m10s elapsed]
aws_db_instance.mysql_db: Still creating... [04m20s elapsed]
aws_db_instance.mysql_db: Still creating... [04m30s elapsed]
aws_db_instance.mysql_db: Creation complete after 4m38s [id=db-GUJ0ZPU3DFXC65TJ6DZBAHKLMM]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:
ec2_public_ip = "3.111.23.5"
load_balancer_dns = "terraform-alb-526289856.ap-south-1.elb.amazonaws.com"
rds_endpoint = "terraform-20260524063321232300000001.cpwai2qo8t98.ap-south-1.rds.amazonaws.com:3306"
PS C:\Users\prasa\OneDrive\Desktop\terraform-aws-project> terraform state list
aws_db_instance.mysql_db
aws_db_subnet_group.db_subnet_group
aws_instance.web_server
aws_internet_gateway.igw
aws_lb.app_lb
aws_lb_listener.listener
aws_lb_target_group.tg
aws_lb_target_group_attachment.tg_attachment
aws_route_table.public_rt
aws_route_table_association.public_assoc_1
aws_route_table_association.public_assoc_2
aws_security_group.web_sg
aws_subnet.private_subnet
aws_subnet.public_subnet_1
aws_subnet.public_subnet_2
aws_vpc.main_vpc
PS C:\Users\prasa\OneDrive\Desktop\terraform-aws-project> |
```

Verify all resources are created successfully in AWS Console.



Cloud Infrastructure Automation using Terraform and AWS

The image displays two screenshots of the AWS Management Console, illustrating the configuration of cloud infrastructure for Terraform.

Top Screenshot: VPC Dashboard

- Page:** VPC dashboard > Your VPCs
- Section:** Your VPCs (1/2)
- Table:** A table listing VPCs. The first entry is 'main-vpc' with VPC ID 'vpc-050e8508218d0ac95', State 'Available', and IP version 'IPv4'.
- Details View:** The 'main-vpc' details are shown, including:
 - VPC ID:** vpc-050e8508218d0ac95
 - DNS resolution:** Enabled
 - Main network ACL:** acl-02dc29d3416382ac0
 - State:** Available
 - Tenancy:** default
 - Default VPC:** No
 - Block Public Access:** Off
 - DHCP option set:** dopt-0cb000e37c39e15cd
 - IPv4 CIDR:** 10.0.0.0/16
 - DNS hostnames:** Enabled
 - Main route table:** rtb-0b0e49c4d6fcd25ac
 - IPv6 pool:** -

Bottom Screenshot: Target Groups

- Page:** EC2 > Target groups
- Section:** Target groups (1/1)
- Table:** A table listing target groups. The first entry is 'terraform-target-group' with ARN 'arn:aws:elasticloadbalancing:ap-south-1:462009753849:targetgroup/terraform-target-group/84d41a747726514f', Port '80', Protocol 'HTTP', Target type 'Instance', Load balancer 'terraform-alb', and VPC ID 'vpc-0238d...'.
- Details View:** The 'terraform-target-group' details are shown, including:
 - Target type:** Instance
 - IP address type:** IPv4
 - Protocol : Port:** HTTP: 80
 - Protocol version:** HTTP1
 - VPC:** vpc-0238dfff5c98e81757
 - Load balancer:** terraform-alb

Cloud Infrastructure Automation using Terraform and AWS

The top screenshot displays the AWS Aurora console for a database instance. The instance is named 'terraform-20260524063321232300000001' and is in an 'Available' state. It is a MySQL Community engine instance in the 'ap-south-1a' region. The CPU usage is shown as 4.66%.

The bottom screenshot displays the AWS EC2 console for the 'Load balancers' page. It shows a single active load balancer named 'terraform-alb' in the 'Active' state. The load balancer is of type 'application' and is 'Internet-facing' with an 'IPV4' address type. It is associated with VPC 'vpc-0238dff5c98e81757' and spans 2 availability zones.

Troubleshooting:

Issue 1: EC2 Apache Service Not Running

Problem:

- Application Load Balancer showed:
- 502 Bad Gateway

Cause: Apache web server was not installed properly.

Solution:

- Connect to EC2 using SSH.
- Install Apache2 manually.
- Start and enable Apache service.

Commands used:

- apt update -y
- apt install apache2 -y
- systemctl start apache2
- systemctl enable apache2

Issue 2: ALB Subnet Error

Problem: Load Balancer required minimum two subnets in different Availability Zones.

Solution: Created two public subnets:

- ap-south-1a
- ap-south-1b

Issue 3: RDS Subnet Group Error

Problem: RDS subnet group did not cover multiple Availability Zones.

Solution: Configured subnets in different AZs for RDS subnet group.

Conclusion:

The Cloud Infrastructure Automation using Terraform and AWS project successfully automated the provisioning of AWS cloud resources using Terraform. The project implemented: Custom VPC, Public and Private Subnets, EC2 Web Server, Application Load Balancer, Amazon RDS MySQL Database. Terraform simplified infrastructure deployment using Infrastructure as Code (IaC), reduced manual configuration effort, and improved deployment consistency. This project demonstrates modern cloud automation and DevOps practices using AWS and Terraform.